What Do AI Agents Actually Change? An Empirical Taxonomy of Mutation Patterns in Performance-Improving Pull Requests

Anonymous

Anonymous Institution

Abstract. AI coding agents are black boxes: we cannot inspect how they generate code, but we can inspect what they change. This distinction matters for search-based software engineering (SBSE), where techniques such as genetic improvement — in the performance-optimisation application we study — depend on mutation operators that reflect how code is actually transformed. Fewer than 1% of the 33,596 agent PRs in AIDev-pop target performance, making each case a rare window into otherwise opaque agent behaviour. We classify 1,254 performance-relevant diff hunks from 216 of these PRs, spanning five agent systems, against the 18-category syntactic mutation taxonomy of Even-Mendoza et al. (2025) using a dual-LLM intersection pipeline. Three categories dominate: *name_modification* (37.0%), *object_creation* (26.4%), and *type_change* (22.7%), a profile markedly different from prior GI corpora where *no_change* accounted for 84%. Each agent's deployed system commits to a distinctive mutation vocabulary, and each performance strategy activates a largely disjoint category subset. Agent identity and target strategy are therefore informative priors that narrow the effective SBSE operator space.
Replication package: `https://anonymous.4open.science/r/ssbse-challenge-2026-710C/`

Keywords: mutation testing · AI agents · empirical study · search-based software engineering · performance optimization

1 Introduction

AI coding agents such as Devin, GitHub Copilot, Cursor, OpenAI Codex, and Claude Code autonomously submit pull requests to production repositories, but their mechanisms for deciding what to change are opaque. We cannot inspect those mechanisms, but we can inspect the outputs: the actual code transformations they commit.

This distinction matters for SBSE. Genetic improvement and related techniques depend on mutation operators grounded in empirical evidence of how code is actually transformed [2,3,7]. We focus specifically on the performance-improvement application of GI. Classical mutation taxonomies were derived from human-written patches. As SBSE is increasingly applied to agent-assisted workflows, an empirical map of how agents actually transform code is a missing

prior: descriptive of agent behaviour for the purpose of scoping operator selection in agent-aware tooling, not prescriptive of agent behaviour as an optimum.

Performance-improving PRs are rare: only 324 of the 33,596 PRs in AIDev-pop [4] carry a performance label (<1%) [5,6], as most agent PRs target bugs or features. These rare cases are the instances where agents intentionally optimised code, making post-hoc mutation analysis directly informative for SBSE.

We address two research questions:

RQ1. *What syntactic mutation patterns characterise successful performance PRs from AI coding agents, and how do they differ from those observed in prior genetic improvement work?*

RQ2. *Do mutation patterns vary systematically across agent systems and across performance strategies?*

2 Dataset and Methodology

2.1 Dataset

We use the AIDev-pop subset of the AIDev dataset [4]: PRs from five AI coding agents (Devin, GitHub Copilot, Cursor, OpenAI Codex, and Claude Code) against 100 starred repositories. Of the 324 PRs carrying a performance label [6], 280 had a retrievable diff; 269 contained at least one source-code hunk after discarding tests, configuration, documentation, and lock files. Each PR's diff is decomposed into contiguous change *hunks*. A first-pass LLM filter (`claude-sonnet-4-6`) retains only performance-relevant hunks (those plausibly affecting runtime, memory, or I/O); cosmetic-only edits and single-model false positives are discarded by the dual-LLM intersection (Sec. 2.3). After agreement filtering, 216 PRs contribute at least one accepted hunk, yielding 1,254 hunks for analysis. Each PR is assigned to one of nine optimisation patterns [6].

2.2 Mutation Taxonomy

We adopt the 18-category syntactic mutation taxonomy of [1], originally derived from manual analysis of LLM-generated patches on Java projects in a Genetic Improvement (GI) pipeline. The categories cover structural transformations (e.g., *control_flow*, *statement_splitting*), identifier-level edits (*name_modification*, *type_change*), and resource-management changes (*synchronization*, *object_creation*), plus two sentinel categories.

Three researchers calibrated category boundaries through six iterative prompt-design rounds on a held-out 40-hunk sample (Section 2.3; full protocol in replication package), most notably extending *object_creation* (cat. 14) to cover import-statement changes, since importing a new library constitutes a module-level creation in the performance context.

Table 1. Mutation taxonomy with corpus frequencies (multi-label; % of 1,254 classified hunks).

ID	Category	Count	%	Description
12	name_modification	463	37.0	Modified variable, function, class name
14	object_creation	330	26.4	Modified object/primitive creation or initialization; new imports
7	type_change	284	22.7	Changed data types or type usage
13	control_flow	262	20.9	Modified control flow: if/else, loops, switch
15	statement_splitting	232	18.5	Split a statement into multiple lines
2	comment_modification	167	13.3	Modified a comment (add/remove/edit)
3	deleted_blocks	90	7.2	Deleted blocks in a method (all/most/some)
16	arithmetic_manipulation	80	6.4	Boolean var. or expr. manipulations
5	return_statement_mods	56	4.5	Add/remove/edit return statements
6	method_name_change	24	1.9	Changes to method names
9	exception_handling	17	1.4	Unreachable or reachable exception
0	added_code_from_github	16	1.3	Added (some arbitrary) code from GitHub
11	synchronization	10	0.8	Added synchronization logic
8	inlining	2	0.2	Replacing a function call with actual body
17	dead_code	1	0.1	Added dead code
4	duplicate_code	1	0.1	Duplicate code
1	no_change	0	0.0	No meaningful code change
10	extra_brackets	0	0.0	Added extra brackets

2.3 LLM-as-a-Judge Classification

We adopt an LLM-as-a-judge approach [8]: each hunk is independently classified by `claude-sonnet-4-6` and `gpt-5.4` using the same structured prompt (18 categories with definitions, examples, and disambiguation notes; cost $22). Prompt accuracy was validated over six design rounds on a 40-hunk sample (uniform random, seeds 42→777, one boilerplate repository excluded); two independent runs confirmed 67.5% exact and 22.5% partial accuracy—substantially above chance for 18-class multilabel. For each hunk the two outputs are compared by category-set intersection: full agreement (48.4%, n=607) adopts the label; partial agreement (51.6%, n=647) takes the intersection; full disagreements are discarded. Category counts are therefore conservative lower bounds.

3 Results

3.1 RQ1: Category Distribution

Table 1 shows the distribution across classified hunks (multi-label). *Name_modification* (37.0%) leads strongly, followed by *object_creation* (26.4%), *type_change* (22.7%), *control_flow* (20.9%), and *statement_splitting* (18.5%). Apriori mining confirms a *name_modification+type_change* co-occurrence cluster (lift 3.07).

The dominance of *name_modification* warrants interpretation. In 78% of *name_modification* cases (Devin PRs following the *Performance-Optimized Dependency Selection* pattern), optimisation manifests as swapping library imports for faster alternatives (e.g., `chalk` → `picocolors`). This makes *name_modification* a *carrier category* whose sub-types (import additions, call-to-cached-variable substitutions, API-level substitutions) warrant separation in future taxonomy revisions. Runtime-risk categories—*synchronization* (0.8%), *inlining* (0.2%), *exception_handling* (1.4%)—are seldom employed as performance optimisations.

Comparison with Even-Mendoza et al. In the original GI corpus, *no_change* dominated at 84% of patches and *type_change* showed the best test-passing rate [1]. Our corpus is markedly different: *no_change* is entirely absent and *arithmetic_manipulation* is rare (6.4%). GI patches are broadly sampled, whereas AI-agent performance PRs are goal-directed commits, so taxonomies calibrated on GI corpora do not transfer directly to agent-generated code.

3.2 RQ2: Variation by Performance Pattern and Agent

Performance patterns. Figure 1 maps mutation categories to AIDev performance patterns. Two associations dominate: *type_change* concentrates in *Data Structure* (253/284; 89%) and *name_modification* in *Build & Infrastructure* (345/463; 75%). *Object_creation* bridges *Data Structure* (103) and *Memory & Locality* (96); *control_flow* co-occurs with *object_creation* in 36.6% of PRs. Within each agent's primary ecosystem (Devin/TypeScript n=461; Copilot/Rust n=219; Codex/Go n=120) one category dominates: *name_modification* 77%, *type_change* 95%, *control_flow* 63%. Cross-language stability is not established (Sec. 4); each agent's profile co-varies with the language mix it targets.

Agent profiles. Table 2 provides a second dimension of variation. Category assignments are broken down by agent (Claude Code excluded due to small sample size, n=16). Four distinct profiles emerge. *Devin* (670 total) is dominated by *name_modification* (361; 54%), reflecting dependency substitution at scale. *GitHub Copilot* (591) is led by *type_change* (224; 38%), consistent with data-structure upgrades. *OpenAI Codex* (574) shows the broadest profile: *control_flow* (136; 24%) and *object_creation* (101; 18%) lead. *Cursor* (184; smallest sample) shows elevated *comment_modification* (41; 22%); since the perf-relevance filter excludes cosmetic-only hunks, this reflects annotations co-occurring with structural edits rather than standalone documentation changes.

Knowing which agent produced the code is thus a complementary, partly correlated prior alongside target strategy for narrowing the operator space.

4 Threats to Validity

Construct. The taxonomy was derived from Java LLM-generated patches and may not capture every mutation type in our multilanguage corpus. We additionally extended *object_creation* to include module-level imports, so its 26.4% rate

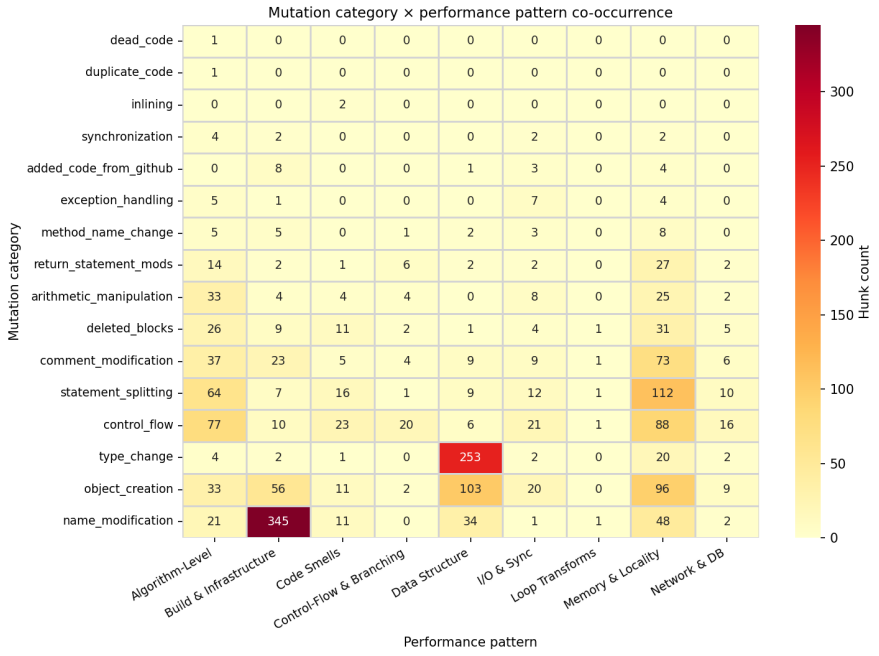


Fig. 1. Co-occurrence of all non-zero mutation categories with AIDev performance patterns. Cells show hunk-category assignment counts (multi-label).

exceeds the original Java-method scope and is not directly comparable to prior corpus statistics. Category boundaries were calibrated for performance PRs and may not generalise to bug-fix or feature PRs. **Internal.** LLM accuracy is bounded at 67.5% after six calibration rounds. Both the first-pass perf-relevance filter and the dual-LLM intersection err toward exclusion, so reported category counts are conservative lower bounds reflecting potential LLM bias [8]; per-category proportions may still be inflated for mutations the filter detects most reliably (e.g., explicit import or rename edits). **External.** Performance labels are content-based: merged (n=324 hunks) and unmerged (n=927 hunks) sub-corpora overlap on 4/5 top categories (Spearman ρ=0.90) with shifted emphases— *control_flow* and *statement_splitting* over-represent merged hunks ($\sim$3×); *name_modification* and *type_change* over-represent unmerged hunks (Cliff's $|\delta| \leq 0.28$); we retain both. All percentages are hunk-weighted, so PRs producing many hunks contribute proportionally more weight than concise PRs. Performance labels themselves originate from [6]; upstream mislabelling would propagate to our scope. Agent and language ecosystem are correlated in AIDev-pop (Devin→TypeScript; Copilot→Rust; Codex→Go), so per-agent profiles describe deployed systems and may not generalise to other agents or repositories. Agent and judge-LLM versions are snapshots; results may shift as these systems evolve.

Table 2. Category assignments per agent (top 8 categories; sorted by agent total).

Category	Devin	Copilot	Codex	Cursor
name_modification	361	30	61	11
object_creation	81	116	101	29
type_change	35	224	22	2
control_flow	51	48	136	24
statement_splitting	65	49	95	21
comment_modification	22	64	37	41
deleted_blocks	13	17	43	15
arithmetic_manip	16	16	29	17
All categories (total)	*670*	*591*	*574*	*184*

5 Conclusion

AI coding agents are black boxes, but their outputs are not. By classifying 1,254 performance-relevant diff hunks we find agent performance PRs dominated by *name_modification*, *object_creation*, and *type_change* — sharply different from prior GI corpora (RQ1) — with each agent and strategy activating a distinct subset (RQ2; agent profiles co-vary with the language mix each system targets, Sec. 4). In a GI loop conditioned on (target strategy, agent), per-context priors from Tables 1–2 narrow the operator space from 18 categories to roughly five.

References

1. Even-Mendoza, K., et al.: LLM-guided genetic improvement: Envisioning semantic aware automated software evolution (2025)
2. Harman, M., Jones, B.F.: Search-based software engineering. Information and Software Technology **43**(14), 833–839 (2001)
3. Jia, Y., Harman, M.: An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering **37**(5), 649–678 (2011)
4. Li, H., et al.: AIDev: A large-scale dataset of AI-generated pull requests (2024), `https://arxiv.org/abs/2412.02551`
5. Opu, M.N.I., et al.: How do agentic AI systems address performance optimizations? a BERTopic-based analysis of pull requests. In: International Conference on Mining Software Repositories (MSR) - Mining Challenge Track (2026), `https://arxiv.org/abs/2512.24630`
6. Peng, H., et al.: How do agents perform code optimization? An empirical study. In: International Conference on Mining Software Repositories (MSR) - Mining Challenge Track (2026), `https://arxiv.org/abs/2512.21757`
7. Petke, J., et al.: Program transformation landscapes for automated program modification using Gin. Empirical Software Engineering **28**(4), 104 (2023)
8. Zheng, L., et al.: Judging LLM-as-a-judge with MT-bench and chatbot arena. In: Advances in Neural Information Processing Systems (2023)